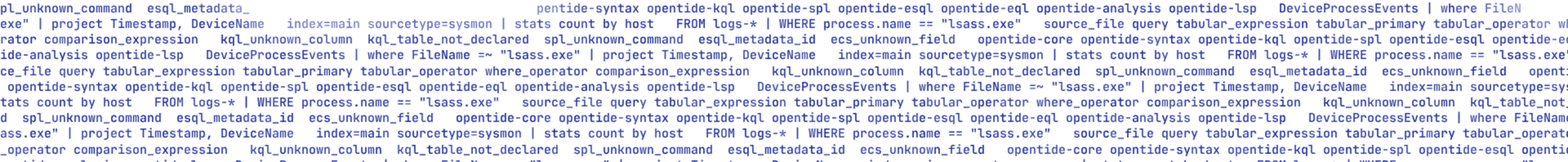
How opentide reads a query.

Inside the language server: a Rust engine that cuts every detection query into tokens, builds a syntax tree, and runs analyzers on it while you type.



Every keystroke, checked.

Between your editor and your detections sits a small, fast program: the language server. It reads the query the way the platform will, and tells you what is wrong before anything runs.

eight languages, one engine

KQL · SPL · ES|QL · EQL · Kibana KQL · Lucene · Falco · Tide YAML

objects/rules/lsass-process-access.yaml — opentide-lsp · stdio

```
1  name: LSASS process access
2  configurations:
3    defender_for_endpoint:
4      query: |
5        DeviceProcessEvents
6        | where FileNam =~ "lsass.exe"
7        | project Timestamp, DeviceName
```

error kql_unknown_column

'FileNam' is not a column of DeviceProcessEvents

Near miss of a catalog column: FileName

✓ diagnostics · hover · completion · semantic tokens · go to definition

opentidehq/language-server · crates/opentide-kql/src/schema.rs · kql_unknown_column

From keystroke to red line.

The same pipeline runs for every language. Hover a step.

01

Text

`edits from the editor`

You type. The editor sends only what changed.

02

Tokens

`kind + byte span`

The text is cut into words the language knows.

03

Syntax tree

`tree-sitter`

Words are grouped by what belongs to what.

04

Analyzers

`catalogs + checks`

Checks ask questions of the tree.

05

Answers

`diagnostics · hover`

Red lines, hovers and suggestions in your editor.

`01 · text` Incremental sync: each keystroke arrives as a small edit, positions counted in UTF-16 the way editors count them.

01 / 03

Cut the text into tokens.

Before it can understand anything, the engine has to know where each word starts, where it ends, and what kind of word it is.



One pass, left to right.

The lexer reads the query once and tags every piece with a kind and its exact position. Spaces and line breaks are skipped. Hover a token.

query · 83 bytes

```
DeviceProcessEvents
| where FileName =~ "lsass.exe"
| project Timestamp, DeviceName
```

token stream · 11 / 11

identifier	"|"	"where"	identifier	"=~"	string	"|"	"project"
DeviceProcessEvents	|	where	FileName	=~	"lsass.exe"	|	project

identifier	","	identifier
Timestamp	,	DeviceName

token 06

"lsass.exe"

kind	string
bytes	40..51
colour	@string

A literal value, kept exactly as written.

02 / 03

Group the tokens into a tree.

Words alone are not meaning. The parser works out which word belongs to which step, the way you parse a sentence into subject and verb.



Every token finds its parent.

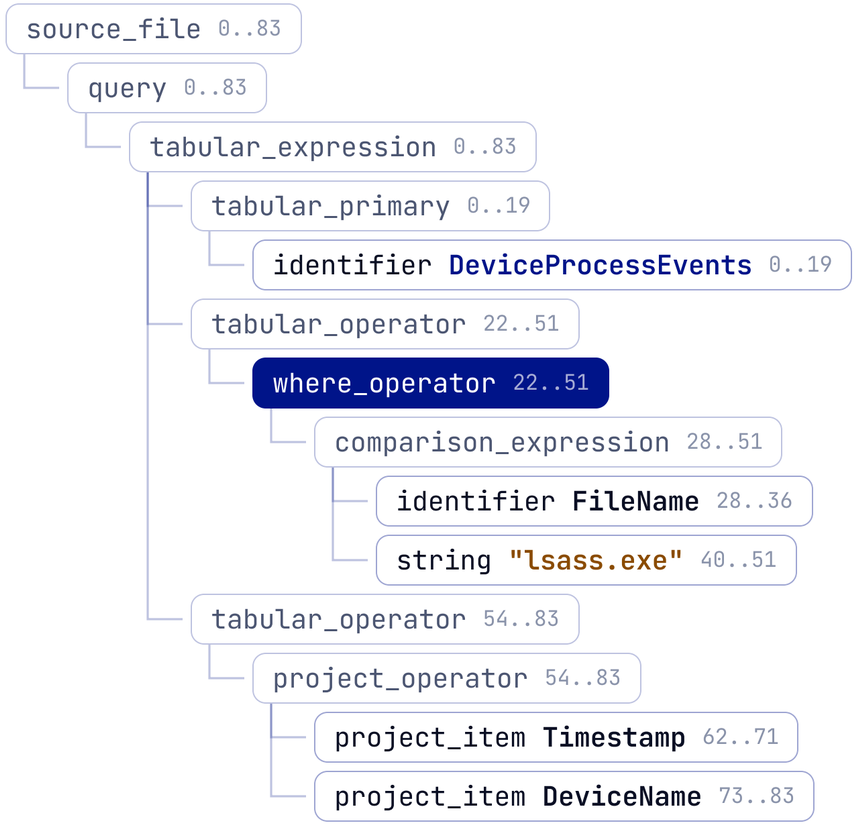
The parser follows the KQL grammar and nests each token inside the step it belongs to. Hover a node: the characters it covers light up.

```
DeviceProcessEvents
| where FileName =~ "lsass.exe"
| project Timestamp, DeviceName
```

`where_operator`

A filter: keep only the rows that match.

concrete syntax tree · named nodes · wrapper chains folded



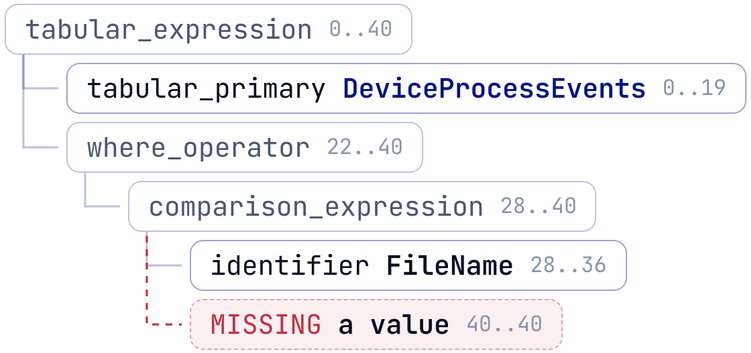
Half-typed is still a tree.

Most of the time you are mid-sentence. The parser marks what is missing and keeps everything else, so the checks never stop.

you are typing… incomplete

```
DeviceProcessEvents
| where FileName =~ |
```

illustrative · the tree the parser keeps



ERROR and MISSING nodes

tree-sitter marks the gap and keeps the rest of the tree intact.

No stack to blow

The tree walk is iterative: a query thousands of levels deep cannot exhaust the stack.

Fuzzed, all nine inputs

kql, spl, esql, eql, kuery, lucene, falco, tide and the protocol. Saved crashes replay in every test run.

03 / 03

Ask the tree questions.

With the structure known, analyzers walk the tree and check it against what your platform really has. Each finding is a named code with an exact range.



One tree, many questions.

Each analyzer asks the same tree one question. Every answer is a named code. Hover one.

the syntax tree

```
tabular_expression
  tabular_primary
    identifier DeviceProcessEvents
  tabular_operator
    where_operator
      comparison_expression
        identifier FileName
        string "lsass.exe"
  tabular_operator
    project_operator
```

error kql_unknown_column

'FileNam' is not a column of DeviceProcessEvents

- **Syntax** — Is the query well formed? — `kql_parse_error`
- **Tables** — Is the table real, and collected here? — `kql_table_not_declared`
- **Columns** — Is every column real at this step? — `kql_unknown_column`
- **Performance** — Does the query filter early? — `kql_where_not_first`
- **Platform** — Is this allowed in a detection? — `kql_control_command_unsupported`
- **Tide objects** — Does the rule point at a real objective? — `invalid_ref`

Checked against what the platform really has.

Table and column catalogs are compiled into the engine. Sentinel alone knows 943 Azure Monitor tables. Pick a typo and see the column check work.

a table name, three outcomes

- ✓ In the catalog and declared in visibility.toml
  `ok`
- ⚠ In the catalog, but not collected here
  `kql_table_not_declared`
- ✕ Not a table opentide knows
  `kql_unknown_table`

Under Sentinel an unknown table is a warning, since a workspace may hold custom tables. Under Defender it is an error.

try **FileNam** | filename | AccountNme | ProcesId | Hostname

DeviceProcessEvents · 14 of 46 catalog columns — edit distance to “FileNam”

Timestamp	5	DeviceId	7
DeviceName	5	ActionType	9
FileName	1	FolderPath	6
ProcessId	8	ProcessCommandLine	16
AccountName	8	AccountDomain	11
InitiatingProcessFileName	18	SHA256	7
MD5	7	FileSize	4

`kql_unknown_column` near miss of `FileName`, offered as the fix

Queries live inside YAML. So does the engine.

A rule’s query is a block of text in a YAML file. The engine lifts it out, strips the indent, analyses it, and maps every byte back, so the red line lands on the right character.

```
1  name: LSASS process access
2  configurations:
3    defender_for_endpoint:
4      query: |
5        DeviceProcessEvents
6        | where FileName =~ "lsass.exe"
7        | project Timestamp, DeviceName
```

defender_for_endpoint.query · KQL, Defender profile

```
DeviceProcessEvents
| where FileName =~ "lsass.exe"
| project Timestamp, DeviceName
```

in the query

bytes 28..36

in the file

line 6, col 15

The same works for SPL in `splunk.query`, ES|QL and EQL in `elastic.query`, Falco in `sysdig.condition`, folded or quoted scalars included.

Rust, and measured.

Compiled to native code, no garbage collector, no runtime to start. Where it was slow, the team measured and fixed it.

Hover, 1,000-object workspace (median) — > 70×



SPL analysis per query — 68×



KQL analysis per query — 4×



How: highlight queries compiled once per process, not on every analysis; the workspace read and indexed once, re-read only when a file changes.

Cargo.toml

```toml
[profile.release]
lto = true            # optimise across every crate
codegen-units = 1     # one unit: best machine code
strip = "symbols"     # smaller binary
```

I/O-free engine

Engine crates never touch the disk. The host hands them text, so the same code runs in a terminal or a browser.

Everything compiled in

Grammars (vendored parser.c) and catalogs (include_str!) ship inside the binary. No downloads at run time.

Only the change

The editor sends each edit, not the whole file.

One engine, everywhere you write detections.

Fifteen crates, one entry point. The same compiled engine answers in the terminal, the editor, the browser and CI.

opentide CLI

`pip install opentide` → `opentide lsp start` (PyO3, abi3 wheel)

Editors

`opentide-lsp --stdio`: VS Code, Neovim, Helix, Zed

entry point

opentide-analysis

core · syntax · highlight · kql · spl · esql · eql · kuery · lucene · falco · ecs · tide · **analysis** · lsp · wasm

Browser

`opentide-wasm` in a Web Worker, no server at all

CI and agents

`opentide-lsp analyze <file>` → plain JSON diagnostics

opentide

Agents write the queries. The spec keeps them aligned.

Tokens, a tree, analyzers: the language server checks every query as it is written, by a person or by an agent.

opentide.org

github.com/opentidehq/language-server

opentide lsp start

